



CSC_42021_EP CONCEPTION & ANALYSE D'ALGORITHMES (2025–2026)

Assignment 3: Path Planning Algorithms

13 Février 2026

Ivann Kamdem
Pacifique Nganta

Coordinateur : Marceau Coupechoux



1

INTRODUCTION

Le but de notre projet était de résoudre un problème de **planification de trajectoire**. L'objectif est de permettre à un robot d'aller d'un point de départ à un point d'arrivée en évitant des obstacles, tout en parcourant la distance la plus courte possible. L'environnement est représenté par un grand rectangle contenant des zones libres et des obstacles rectangulaires. Le robot part d'une position initiale et doit atteindre une position finale sans entrer en collision avec ces obstacles.

Ce problème est intéressant car l'espace dans lequel on cherche la solution est très grand ainsi un chemin peut passer par une infinité de points intermédiaires, ce qui rend le nombre de solutions possibles très grand. Ensuite, la présence d'obstacles complique l'optimisation, car un chemin peut être court mais invalide s'il traverse un obstacle. Cela rend la fonction à minimiser (la longueur du chemin) assez complexe. Les méthodes classiques d'optimisation ne suffisent donc pas toujours.

Pour résoudre ce problème, nous avons étudié deux approches différentes. La première est le **Particle Swarm Optimization (PSO)**. On considère plusieurs "particules", chacune représentant un chemin possible. À chaque étape, chaque particule améliore sa solution en tenant compte de sa meilleure solution passée et de la meilleure solution trouvée par le groupe. Petit à petit, l'ensemble converge vers un bon chemin, à la fois court et sans collision.

La seconde méthode est le **Rapidly-exploring Random Tree (RRT)**. Ici, on construit progressivement un arbre à partir du point de départ. À chaque étape, on choisit un point au hasard dans l'espace, puis on essaie de le relier à l'arbre déjà construit, si cela ne traverse pas d'obstacle. L'arbre grandit ainsi dans différentes directions jusqu'à atteindre le point d'arrivée. Cette méthode est particulièrement efficace pour explorer rapidement l'espace libre.

Nous avons aussi étudié des améliorations pour rendre ces méthodes plus performantes. Pour le PSO, par exemple, on peut réinitialiser certaines particules pour éviter qu'elles restent bloquées dans une mauvaise solution. Pour le RRT, on peut améliorer le chemin trouvé à la fin ou générer plus intelligemment les points tirés au hasard.

Enfin, il a été question de résoudre le problème dans un cas plus complexe : celui de deux robots qui se déplacent en même temps. Chaque robot doit éviter les obstacles mais aussi garder une distance de sécurité avec l'autre robot. Cela rend le problème plus difficile, car il faut éviter toute collision.

En résumé, ce projet nous a permis de comparer deux méthodes de planification de trajectoire et d'analyser leur efficacité dans différents scénarios, y compris dans le cas de plusieurs robots.

2

ARCHITECTURE LOGICIELLE ET MODÉLISATION

L'une des spécificités de ce projet est d'avoir dépassé le stade de simples scripts pour proposer une architecture logicielle orientée objet, modulaire et installable via le gestionnaire de dépendances moderne python : `uv`.

2.1 STRUCTURE DU PACKAGE RANDOM-EXPLORER

- Le code source est organisé dans `src/random_explorer/` pour garantir la séparation des responsabilités :
- `environment.py` : Centralise la logique géométrique. Il gère le parsing des scénarios et expose une API unifiée de détection de collision utilisée par tous les solveurs, au moyen d'une classe éponyme.
 - `pso/` : Ce sous-package implémente une classe de base `PSOPathPlanner`, dont héritent les variantes (`PSORestart`, `PSOSimulatedAnnealing`, `PSODimensionalLearning`, ...). Cela permet de réutiliser le moteur physique tout en surchargeant uniquement la logique de mise à jour des particules.
 - `rrt/` : Contient l'implémentation du `RRT*` (`RRTPlanner`), de la version multi-robot (`MultiRobotRRTPlanner`) et des structures de données arborescentes dans `node.py`.
 - `benchmark.py` : Un moteur de tests automatisés, définissant une classe `Benchmark` qui abstrait la logique de réalisation de benchmarks entre les variantes PSO et celles RRT; il permet aussi la visualisation graphique des résultats.

L'utilisation intensive de la bibliothèque `numpy` permet de vectoriser les opérations (calculs de distances matriciels pour le PSO), garantissant des performances élevées même avec de grandes populations.

2.2 MODÉLISATION DE L'ENVIRONNEMENT ET COLLISIONS

L'environnement est un espace continu $[0, L] \times [0, H]$ peuplé d'obstacles rectangulaires. La détection de collision étant l'opération la plus coûteuse, nous avons implémenté deux approches dans la classe `Environment` :

1. **Mode Hard (Discrétisation)** : Le segment est échantillonné en k points, et on vérifie juste si ces points sont dans l'obstacle. Rapide pour l'affichage, mais imprécis si le pas d'échantillonnage est trop grand.
2. **Mode Soft (Analytique)** : Nous utilisons l'algorithme de clipping de **Liang-Barsky** [3]. Contrairement à la méthode de Cohen-Sutherland, Liang-Barsky utilise une représentation paramétrique du segment $P(t) = P_1 + t(P_2 - P_1)$ pour trouver analytiquement les points d'entrée et de sortie dans l'obstacle.

Cette méthode est en $\mathcal{O}(1)$ par obstacle (indépendante de la longueur du segment) et fournit la **profondeur de pénétration**, métrique cruciale pour guider le PSO hors des obstacles via une fonction de coût continue.

3

PARTICLE SWARM OPTIMIZATION (PSO)

Le PSO (Kennedy et Eberhart, 1995 [1]) est une méthode d'optimisation stochastique globale. Une solution est modélisée par une particule représentant un chemin complet de N waypoints : $X_i = [w_1, w_2, \dots, w_N]$.

3.1 DYNAMIQUE ET VARIANTES

La mise à jour de la vitesse V_i et de la position X_i suit l'équation fondamentale :

$$V_i^{t+1} = \omega V_i^t + c_1 r_1 (P_i^{\text{best}} - X_i^t) + c_2 r_2 (G^{\text{best}} - X_i^t) \quad (1)$$

$$X_i^{t+1} = X_i^t + V_i^{t+1} \quad (2)$$

Pour optimiser les hyperparamètres (ω, c_1, c_2) , nous avons réalisé un *grid search orienté*, centré sur une zone de valeurs intuitivement pertinentes : un ω élevé (0.8) pour favoriser l'exploration initiale, des coefficients c_1 et c_2 légèrement supérieurs à ω pour maintenir l'équilibre exploration–exploitation, un nombre important de particules (environ 300) et un nombre d'itérations limité (100) compte tenu des plateaux rapides observés. Cette exploration ciblée nous a permis d'identifier un ensemble de paramètres performants : `num_particles = 300`, `num_waypoints = 8`, `max_iter = 100`, $\omega = 0.8$, $c_1 = 1.6$ et $c_2 = 1.0$. L'objectif était de sonder le voisinage de configurations théoriquement raisonnables plutôt que de balayer tout l'espace des paramètres.

Nous avons implémenté des variantes avancées pour contrer la stagnation :

- **Random Restart** : Réinitialise périodiquement les particules ayant le plus mauvais score, injectant de la diversité dans l'essaim pour échapper aux optima locaux.
- **Simulated Annealing (SA)** : Accepte parfois des solutions moins bonnes avec probabilité $P = \min(1, \exp(-(f(s) - f(g^k))/T^k))$, où la température $T^k = T_0 \beta^k$ décroît avec les itérations. Permet d'explorer plus largement l'espace de recherche.
- **Dimensional Learning (DL)** : Permet à une particule stagnante d'apprendre indépendamment sur chaque dimension (x ou y) d'un waypoint en remplaçant sélectivement les dimensions de sa meilleure position locale par celles de la meilleure position globale. Efficace dans les passages étroits alignés sur les axes, mais plus coûteux en temps.
- **Adaptive Inertia (notre amélioration)** : Fait décroître linéairement le poids d'inertie ω de 0.9 à 0.4, favorisant l'exploration au début et l'exploitation à la fin. Combiné avec un mécanisme d'*early stopping* qui arrête l'algorithme dès qu'un chemin sans collision est trouvé. Efficace pour trouver le bon chemin dans des scénarios complexes avec un bon compromis qualité/temps.

3.2 FONCTION DE COÛT

La fitness $J(X)$ est une somme pondérée : $J(X) = L(X) + P_{\text{base}} \cdot \mathbf{1}_{\text{collision}} + \lambda \cdot \sum \text{overlap}(s_k, \text{obs})$, où $L(X)$ est la longueur géométrique du chemin, $\mathbf{1}_{\text{collision}}$ est un indicateur binaire valant 1 si une collision existe, et $\text{overlap}(s_k, \text{obs})$ représente la métrique de collision. Le terme P_{base} (pénalité de base, typiquement 1000) garantit qu'un chemin avec collision aura toujours un score supérieur à un chemin sans collision, créant une barrière claire entre solutions valides et invalides. En mode "Soft", le terme $\lambda \cdot \sum \text{overlap}(s_k, \text{obs})$ utilise une pénalité proportionnelle à l'intersection calculée par l'algorithme de Liang-Barsky, créant un "champ de potentiel" virtuel repoussant les particules hors des obstacles avec une force proportionnelle à la profondeur de pénétration.

4

RRT ET RRT*

Contrairement au PSO qui déforme un chemin existant, le RRT* (Karaman et Frazzoli, 2011 [2]) explore l'espace libre en construisant un graphe.

4.1 ALGORITHME ET OPTIMISATIONS

L'algorithme itère les étapes suivantes : 1. **Échantillonnage Intelligent** : Au lieu d'un tirage uniforme pur, nous utilisons un **Goal Bias** (5% de chance de viser le but) et un **Obstacle Bias** (tirage proche des

obstacles) pour favoriser la découverte de passages étroits. 2. **Steering** : Extension de l'arbre depuis le nœud le plus proche x_{nearest} vers x_{rand} d'une distance maximale δ_s . 3. **Recâblage** : C'est l'étape la plus importante du RRT*. On recherche les voisins dans un rayon $\gamma(\frac{\log n}{n})^{1/d}$. Si passer par un nouveau nœud réduit le coût cumulé d'un voisin, on met à jour son parent. Cela garantit l'optimalité asymptotique.

Post-traitement : Le chemin brut du RRT est souvent en zigzag. Nous appliquons un algorithme glouton qui tente de relier n_i à n_{i+k} tant que le segment est libre, réduisant considérablement la longueur finale (visible sur le Scénario 1).

Échantillonnage intelligent : Une autre méthode d'amélioration est de générer les points autour des obstacles afin de s'assurer d'atteindre rapidement le résultat final.

5 RÉSULTATS EXPÉRIMENTAUX

Les tests (Table 1) réalisés sur 5 scénarios contrastés révèlent une dichotomie claire entre les deux approches.

TABLE 1 – Comparaison des performances (Moyenne sur les runs valides)

Scénario	PSO*		RRT*	
	Longueur	Temps (s)	Longueur	Temps (s)
0 (Simple)	1445,90	1,63	1447,42	13,78
1 (Passage étroit)	1965,97	3,16	1458,47	10,03
2 (Moyen)	1656,70	6,44	1514,44	7,63
3 (Complexe)	1654,73	13,36	1550,57	7,03
4 (Labyrinthe)	2228,36	9,59	1986,88	4,92

Analyse des performances : Le **PSO** est extrêmement efficace sur le *Scénario 0* (vide), convergeant en 1.63s. Cependant, sa complexité temporelle dépend du nombre d'obstacles M ($\mathcal{O}(I \cdot S \cdot N \cdot M)$). Sur le *Scénario 4* (Labyrinthe), le PSO souffre : il doit évaluer de nombreuses collisions et peine à "plier" le chemin pour entrer dans les couloirs, résultant en un taux d'échec plus élevé sans *Random Restart*.

Le **RRT*** montre une robustesse supérieure. Sa performance dépend principalement de la complexité topologique plutôt que du nombre d'obstacles géométriques, grâce aux structures spatiales. Bien qu'il soit initialement plus lent sur les cas simples, il surpasse le PSO sur les cas complexes (Scénario 4 : 4.92s contre 9.59s), car il "découvre" la topologie au lieu de tenter de l'optimiser aveuglément, **comme illustré par les trajectoires vertes sur la Figure 1**.

6 EXTENSION MULTI-ROBOTS

Nous avons implémenté une planification découplée pour deux robots.

1. **Priorité** : Le Robot 1 planifie sa trajectoire en considérant le Robot 2 comme un obstacle statique.
2. **Espace-Temps** : Le Robot 2 planifie ensuite en considérant la trajectoire du Robot 1 comme une contrainte dynamique spatio-temporelle.
3. **Vérification** : Une fonction `check_inter_robot_collision` valide que la distance $\|R_1(t) - R_2(t)\| > 2R$ à tout instant t .

Cette méthode est efficace (linéaire en nombre de robots) pour des environnements ouverts, mais peut mener à des situations de blocage dans les couloirs étroits, situation que nous avons observée expérimentalement et qui nécessiterait une planification conjointe dans l'espace de configuration composite $\mathcal{C}_{\text{free}}^1 \times \mathcal{C}_{\text{free}}^2$.

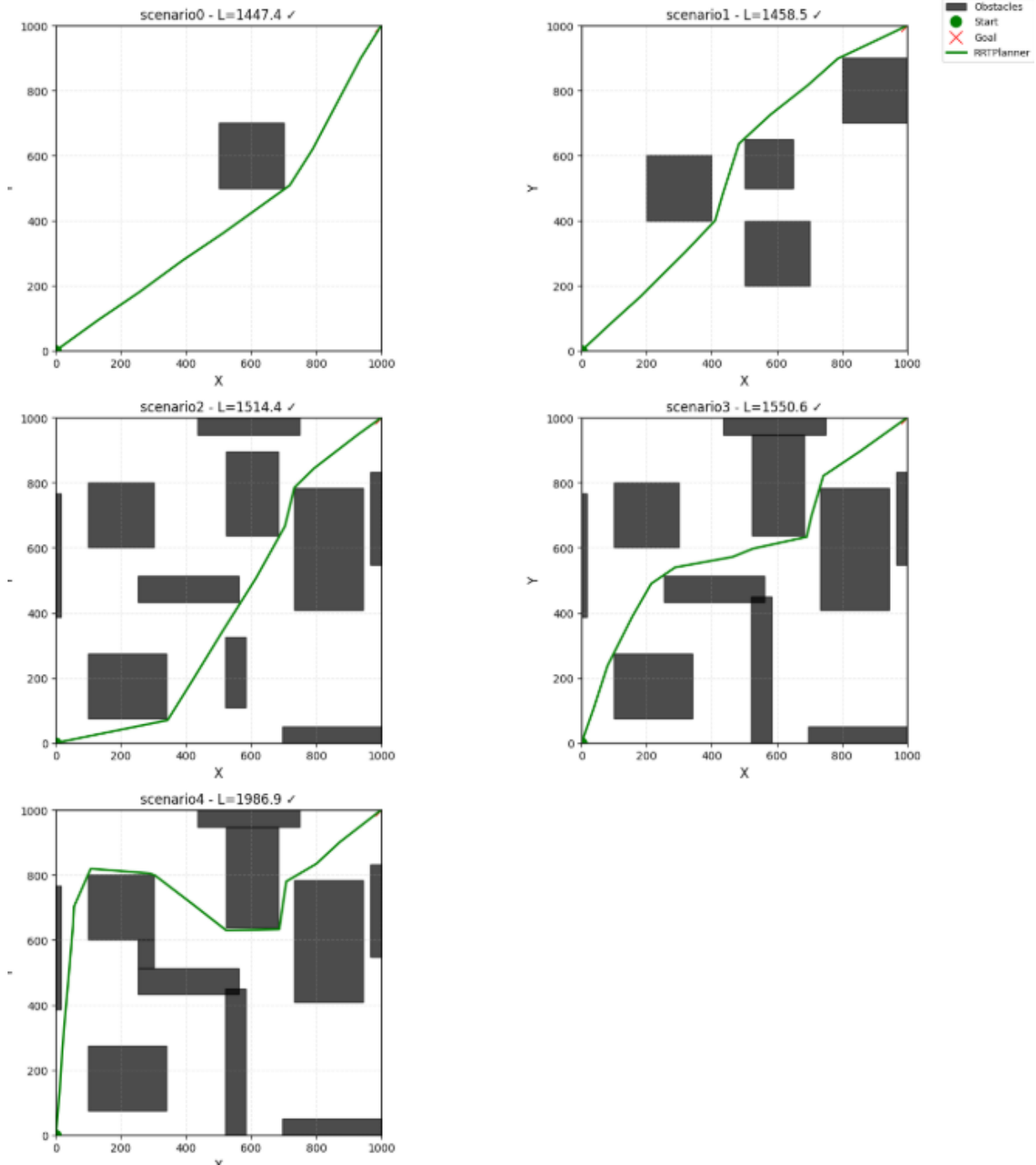


FIGURE 1 – Trajectoires générées par RRT* sur les cinq scénarios de test. On constate que l'algorithme réussit systématiquement à trouver un chemin valide (en vert), même dans les configurations difficiles comme le labyrinthe (scénario 4) ou le passage simple (scénario 1).

CONCLUSION ET PERSPECTIVES

Ce projet de conception et d'analyse d'algorithmes nous a permis de confronter deux méthodes radicalement différentes de la planification de trajectoire : l'optimisation continue par un ensemble de particules (**PSO**) et l'exploration probabiliste discrète (**RRT***). Au-delà de l'implémentation algorithmique, la réalisation du package *random-explorer* a mis en lumière l'importance d'une architecture logicielle modulaire pour mener des benchmarks rigoureux.

Nos résultats expérimentaux (notamment sur le Scénario 4 de type labyrinthe) permettent de dresser un bilan clair des avantages de chaque algorithmes :

- **Le PSO** : Le PSO se comporte comme un ensemble de particules cherchant des optimums à suivre. Sur des espaces convexes ou peu encombrés, il est imbattable en rapidité et précision. Cependant, il souffre d'une limitation structurelle majeure : il ne possède pas de mémoire de la topologie. Face à un obstacle en forme de U, l'ensemble des particules restent coincées dans un minimum local, nécessitant des améliorations comme le *Random Restart* pour espérer trouver la sortie.
- **Le RRT*** : À l'inverse, le RRT* construit explicitement une carte de la connectivité de l'espace libre. Sa force réside dans sa *complétude probabiliste* : avec suffisamment de temps, il trouvera toujours un chemin s'il en existe un. Bien que plus lent à converger vers l'optimum pur que le PSO, il est permis tout de même de traverser de manière fiable des passages étroits ou des enchaînements complexes ("tunnels") d'obstacles.

Notre extension multi-robots a montré les limites de l'approche découplée. En planifiant les robots l'un après l'autre, nous résolvons efficacement les conflits simples, mais nous créons des situations d'interblocage inévitables dans les couloirs étroits. Pour aller plus loin, il serait nécessaire de :

1. Planifier directement dans l'**espace de configuration joint** ($\mathcal{C}_{free} \subset \mathbb{R}^4$), bien que cela augmente exponentiellement la complexité.
2. Hybrider les méthodes : utiliser un RRT grossier pour identifier la topologie du chemin, puis utiliser le PSO pour optimiser finement la trajectoire à suivre pour résoudre le problème.

Si l'on prend du recul sur ces algorithmes, on peut les classer selon leur rapport à l'espace et au coût. Le RRT peut être vu comme une version stochastique et discrète de l'algorithme de **Dijkstra** : il explore un graphe construit de façon progressive.

Une perspective fascinante serait d'explorer l'approche continue de ce problème via les **Équations Eikoniales** (résolues par la *Fast Marching Method*).

- **L'utilisation de la lumière** : Là où Dijkstra propage un coût de nœud en nœud, l'équation Eikonale $|\nabla T(x)| = 1/F(x)$ modélise la propagation d'un front d'onde (comme la lumière ou le feu) depuis le départ.
- **Dijkstra Continu** : Cette méthode permet de gérer des environnements hétérogènes. Contrairement à nos obstacles binaires (0 ou 1), l'Eikonale permet de définir des zones "boueuses" ou "sablonneuses" (coût de traversée élevé mais fini). Le chemin optimal devient alors une géodésique courbée par le "milieu", analogue à la réfraction de la lumière (Loi de Snell).

Ainsi, on peut constater que l'algorithme RRT fonctionne très bien dans une exploration de grands espaces, et le PSO quant à lui est efficace dans un ajustement local, tandis que l'approche Eikonale offrirait une solution globale déterministe et physiquement cohérente pour la robotique en milieu naturel complexe.

RÉFÉRENCES

- [1] Kennedy, J. and Eberhart, R. (1995). Particle Swarm Optimization. *Proc. IEEE Int. Conf. on Neural Networks*.
- [2] Karaman, S. and Frazzoli, E. (2011). Sampling-based Algorithms for Optimal Motion Planning. *Int. Journal of Robotics Research*.
- [3] Liang, Y. D. and Barsky, B. A. (1984). A New Concept and Method for Line Clipping. *ACM Trans. on Graphics*.